

Agent Concepts and MCP Basics

Ahmed Elgharably — FlyRank ML Internship — General AI Fluency, Week 4 (FL-05)

Part 1: Workflow vs. Agent — In My Own Words

The Difference

A **workflow** is a system where an LLM moves through a fixed sequence of steps that a human designed in advance. The path is deterministic. Step 1 always feeds into Step 2. The LLM doesn't decide what to do next — the code does. The LLM is a component inside a machine that a human built.

An **agent** is a system where an LLM dynamically directs its own process. It decides which tool to use, when to use it, and whether the result is good enough to move forward or if it needs to try again. The LLM is the driver, not a passenger. It maintains a loop: observe → decide → act → check → repeat.

The distinction is not about complexity. A workflow can have 50 steps and an agent can have 3. The distinction is **who controls the path**.

	Workflow	Agent
Path control	Human (predefined code)	LLM (dynamic decisions)
Tool use	Fixed: Step 3 always calls Tool X	Dynamic: LLM decides which tool, when, and why
Error handling	Human-coded fallback branches	LLM self-corrects and retries
Predictability	High —same input, same path	Lower —same input, different paths possible
Debuggability	Easy —trace the code	Harder —trace the LLM's reasoning
When to use	Tasks with clear, repeatable steps	Tasks with ambiguous goals and unpredictable obstacles

Classifying My FL-04 Pipeline

My FL-04 pipeline is a **workflow**, not an agent.

Here's why: every step is predetermined. I upload a PDF to NotebookLM. I ask the same 6 questions every time. I paste the answers into Claude with the same formatter prompt. I review the output with the same checklist. The LLM never decides "this paper is too short, skip the synthesis step" or "NotebookLM failed, try a different tool." I make those calls. The LLM is a component inside my 4-step machine.

An agent version would look different. It would receive a paper URL, decide whether to download it, choose between NotebookLM or direct PDF parsing based on file size, determine how many questions to ask based on paper length, decide whether the output is high-quality enough to format or if it needs re-synthesis, and choose where to save the final note based on topic. The LLM would be the orchestrator. In my current pipeline, I am.

Part 2: What Is MCP?

MCP (Model Context Protocol) is an open standard that lets AI applications connect to external tools, data sources, and services through a unified interface. Anthropic describes it as a “USB-C port for AI applications” — one standardized plug that works with many devices.

Before MCP, every AI tool integration was custom. If you wanted Claude to read your local files, you built a file reader. If you wanted it to query a database, you built a database connector. Each integration was a one-off hack. MCP replaces that chaos with a protocol: any tool that speaks MCP can plug into any AI client that speaks MCP.

The Three Primitives

1. Tools — Functions the model can call to perform actions.

A tool is an action the LLM can invoke. It has a name, a description, and a schema defining its inputs. When the LLM decides it needs to do something — search the web, read a file, run a calculation — it calls the tool with the right parameters. The tool executes, returns a result, and the LLM uses that result to continue.

Example from my world: a `read_pdf` tool that takes a file path and returns the extracted text. Or a `search_arxiv` tool that takes a query and returns matching papers.

2. Resources — Data the model can read as context.

A resource is read-only data that the model can access. Unlike tools, resources don’t perform actions — they expose information. The model can read them to ground its responses in real, current data.

Example: my `research-notes/` folder exposed as a resource. The model can read my existing notes to avoid duplication or to build on previous work. Or my `dim_clients` table exposed as a resource so the model knows which clients have enough history for seasonality analysis.

3. Prompts — Pre-defined templates that guide the model’s behavior.

A prompt is a reusable template that the model or the user can invoke. It standardizes how the model approaches a task. Instead of typing the same long instructions every time, you define a prompt once and call it by name.

Example: my “Research Note Formatter” prompt from FL-04, exposed as an MCP prompt. Instead of pasting the full template into Claude every time, I invoke `/format-research-note` and the model knows the structure, voice, and rules.

Part 3: MCP Setup — Three Tasks Chat Alone Could Not Do

I connected the **Filesystem MCP Server** to Claude Desktop. This server exposes my local file system as tools and resources that Claude can access directly.

How I Set It Up

1. **Installed Claude Desktop** (already had it)
2. **Installed the Filesystem MCP Server** via npm:

```
npm install -g @modelcontextprotocol/server-filesystem
```

3. **Configured Claude Desktop** to use it by editing `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/ahmed/flyrank-ml-internship"
      ]
    }
  }
}
```

4. **Restarted Claude Desktop** — the server appeared as a connected tool

Task 1: Read My Existing Research Notes Before Writing a New One

What I asked: “I just read a new paper about CTR prediction. Before I write a note, check my existing research notes and tell me if I’ve already covered something similar.”

What chat alone could not do: Chat has no access to my local files. It doesn’t know what papers I’ve already read or what notes I’ve written. Without MCP, I’d have to manually open each file, read it, and compare.

What MCP did: Claude called the `read_file` tool on every `.md` file in `research-notes/`, scanned the topics, and reported: “You already have a note on ‘Deep Learning for Click-Through Rate Prediction’ from July 2026. The new paper uses a similar architecture (DeepFM) but adds temporal features. Consider updating the existing note rather than creating a duplicate.”

Screenshot evidence: [Claude interface showing tool call: `read_file` on `research-notes/2026-07-15-deep-learning-ctr.md`, then synthesized comparison]

Task 2: Append a New Finding to an Existing Note Without Opening an Editor

What I asked: “Append this takeaway to my BERT note: ‘New experiment: fine-tuned BERT on FlyRank content metadata for intent classification. Accuracy 0.74 vs. TF-IDF baseline 0.61.’”

What chat alone could not do: Chat can draft text, but it cannot write to my file system. Without MCP, I’d copy the drafted text, open VS Code, find the file, scroll to the bottom, paste, and save.

What MCP did: Claude called `read_file` to load the BERT note, found the end of the “FlyRank Takeaway” section, and called `write_file` to append the new experiment result in the correct markdown format. It preserved the existing structure and added a timestamp.

Screenshot evidence: [Claude interface showing tool calls: `read_file` → `write_file`, then confirmation message with diff view]

Task 3: Generate a Summary of All My Notes by Topic

What I asked: “Look at all my research notes and generate a table summarizing what I’ve read by topic: ranking, CTR, content optimization, and embeddings. Include the best paper per topic and one practical takeaway.”

What chat alone could not do: Chat has no memory of my past conversations about papers, and it cannot browse my local files. Without MCP, I’d have to manually open 15+ files, extract the topic and takeaway from each, and organize them into a table myself.

What MCP did: Claude called `list_directory` on `research-notes/`, then `read_file` on each `.md` file (15 files), extracted the topic tags and FlyRank Takeaways, and generated:

Topic	Papers Read	Best Paper	Practical Takeaway
Ranking	4	Learning to Rank (ICML 2007)	Listwise loss beats pairwise for top-K metrics
CTR	3	Deep Learning CTR (arXiv 2021)	DeepFM handles feature interactions well; try on FlyRank query-content pairs
Content Optimization	2	Content Refresh Strategies	6-month refresh cycle is the sweet spot for SEO
Embeddings	3	BERT (NAACL 2019)	Fine-tuning on domain text beats generic embeddings for intent classification

Screenshot evidence: [Claude interface showing `list_directory` tool call, multiple `read_file` calls, and final generated table]

Part 4: Explainer — What My FL-04 Workflow Would Need to Become an Agent

My FL-04 pipeline is a workflow. To turn it into an agent, I would need to give the LLM three things it doesn’t currently have: **autonomy, tool access, and a self-correction loop.**

1. Autonomy: Let the LLM Decide the Path

Right now, the path is fixed: Upload → 6 Questions → Format → Review. An agent version would let the LLM decide:

- **Should I read this paper at all?** The agent checks the abstract first. If the paper is theoretical with no empirical results, the agent skips it and reports: “No actionable data — skipped.”
- **How deep should I go?** A 4-page workshop paper gets 3 questions. A 12-page ICML paper gets 8 questions. The agent decides based on length and density.
- **Which tool should I use?** If NotebookLM is down, the agent switches to direct PDF parsing. If the paper is behind a paywall, the agent tries arXiv first, then Semantic Scholar, then reports failure.

The key shift: **I stop writing the if-then branches. The LLM writes them at runtime.**

2. Tool Access: Give the LLM Hands

My current pipeline uses two tools (NotebookLM and Claude) but I am the glue between them. I copy from A and paste into B. An agent version would use MCP to let the LLM call tools directly:

- **search_arxiv tool:** The agent receives a topic like “CTR prediction for content ranking” and searches arXiv itself, picking the 3 most relevant papers without me hunting URLs.
- **read_pdf tool:** The agent downloads the PDF and extracts text directly, no upload step needed.
- **write_note tool:** The agent writes the formatted note to `research-notes/` directly, in the correct markdown structure.
- **query_notebook tool:** The agent sends questions to a local notebook interface and reads the answers.

With MCP, these tools plug into Claude through a standard protocol. I don’t build custom integrations. I install the server, define the tools, and the agent uses them.

3. Self-Correction: Let the LLM Check Its Own Work

My current pipeline has a human review step (Step 4). An agent would add an internal review loop:

- After synthesizing, the agent reads its own summary and asks: “Did I capture the main method or did I focus on a minor detail?”
- After formatting, the agent checks: “Do these numbers match the source, or did I hallucinate the baseline score?”
- If the agent detects a mismatch, it retries: re-reads the source section, re-synthesizes, and checks again.

This is the **agent loop**: not just one pass, but observe → act → verify → retry until a quality threshold is met.

4. What I Would Actually Build (Concrete Agent Upgrade)

Here’s the specific agent upgrade I would implement for my FL-04 pipeline:

The Goal: “Keep me current on ML research relevant to content refresh scoring. Every Monday morning, give me a digest of 3 papers I haven’t read, with summaries and FlyRank takeaways, already saved to my repo.”

The Agent Architecture:

```
Trigger: Every Monday at 9 AM (or on demand)
|
▼
[Step 1: SEARCH] — Agent calls `search_arxiv` with query:
  "content refresh SEO machine learning ranking CTR prediction 2024 2025"
|
▼
[Step 2: FILTER] — Agent reads abstracts of top 10 results,
  compares against existing notes (via `list_directory` + `read_file`),
  picks 3 new papers not already covered
|
▼
[Step 3: DOWNLOAD] — Agent calls `download_pdf` for each paper
```

```
|
▼
[Step 4: SYNTHESIZE] — Agent reads each PDF, asks dynamic questions
based on paper type (theory vs. empirical vs. survey)
|
▼
[Step 5: FORMAT] — Agent invokes the `format-research-note` MCP prompt
|
▼
[Step 6: SELF-CHECK] — Agent verifies key numbers against source,
flags suspicious claims with ⚠️
|
▼
[Step 7: SAVE] — Agent calls `write_file` to save each note to
`research-notes/YYYY-MM-DD-title.md`
|
▼
[Step 8: DIGEST] — Agent generates a Monday morning summary:
"3 new papers processed. 1 directly relevant to Lane 2 (refresh scoring).
1 relevant to Lane 4 (CTR). 1 interesting but not actionable yet."
```

What makes this an agent, not a workflow:

- The agent decides **which papers to read** (not me)
- The agent decides **how many questions to ask** per paper (not a fixed 6)
- The agent decides **which tool to use** based on availability (NotebookLM, direct PDF, or arXiv API)
- The agent **self-corrects** when numbers don't match
- The agent **saves its own output** without me copy-pasting

What I would still review:

- The final digest before it goes to my repo (I don't want garbage saved)
- The FlyRank Takeaway — only I know what my capstone actually needs
- Any ⚠️ flags the agent raised

Why I haven't built this yet:

- It requires Claude Desktop (or another MCP client) running locally with persistent access to my file system
- It needs an arXiv MCP server (exists but requires setup)
- It needs error handling for network failures, paywalls, and malformed PDFs
- The debugging surface is larger — when the agent picks a bad paper, I need to trace why

But the payoff is real: Instead of spending 30 minutes per paper × 3 papers = 90 minutes every week, I spend 5 minutes reviewing the agent's digest. That's 85 minutes back for actual ML work. And the agent doesn't skip weeks because it's "too busy" or "forgot."

Summary

	My FL-04 Workflow	Agent Upgrade
Path	Fixed 4 steps	Dynamic —LLM decides
Tool use	I copy-paste between tools	LLM calls tools via MCP
Paper selection	I find and choose	Agent searches and filters
Quality check	Human review (Step 4)	Agent self-check + human review
Output	I save manually	Agent saves automatically
Time	10–15 min/paper	1–2 min review/paper
Risk	Low —predictable	Higher —needs guardrails

The agent is not better because it’s more complex. It’s better because it handles the decisions I don’t need to make (which papers, which tools, where to save) so I can focus on the decisions only I can make (is this relevant to my capstone, is this claim trustworthy, does this takeaway make sense).

MCP is what makes the upgrade possible. Without a standard protocol, I’d be writing custom Python scripts to connect Claude to arXiv, to my file system, to my notebooks. With MCP, I install three servers and the agent speaks to all of them through one language.

Submitted to the General AI Fluency track thread.